

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
FACULTY OF LIBERAL ARTS AND BUSINESS STUDIES
DEPARTMENT OF COMPUTER SCIENCE



PRACTICAL RECORD

STUDENT NAME :

REGISTRATION NO. :

CLASS : B.SC Computer Science - A

SEMESTER & YEAR : V Semester & III Year

SUBJECT CODE : USC24D04J

SUBJECT TITLE : MACHINE LEARNING

OCT 2026



FACULTY OF LIBERAL ARTS AND BUSINESS STUDIES

DEPARTMENT OF COMPUTER SCIENCE

BONAFIDE CERTIFICATE

This is to certify that this is a bonafide record of practical work done by

NAME :

REGISTER NUMBER :

CLASS & SEC : B.SC Computer Science - A

DEPARTMENT : COMPUTER SCIENCE

SEMESTER & YEAR : V SEMESTER & III YEAR

SUBJECT CODE : UCS24D04J

SUBJECT : MACHINE LEARNING

During the Academic Year 2026 – 2027. Submitted for B.Sc Degree practical examination held on _____.

FACULTY INCHARGE

H.O.D

INTERNAL EXAMINER

EXTERNAL EXAMINER

INDEX

Exp. no.	Date	Experiment	Signature
1		Sum of all individual digits of a given integer using a while loop.	
2		Generate and display the Fibonacci series up to n terms using a for loop.	
3		Check whether a given integer is an Armstrong number.	
4		Check whether a given integer is a Palindrome number.	
5		Check whether a given integer is a Perfect Number.	
6		Convert a given decimal (base-10) integer into its equivalent binary.	
7		Performs element-wise addition of two 1D NumPy arrays.	
8		Calculates the dot product (scalar product) of two vectors.	
9		Performs element-wise addition of two 2×2 matrices.	
10		Performs matrix multiplication (dot product of matrices) on two 2×2 matrices	
11		Finds and displays the transpose of a given matrix using NumPy's attribute.	
12		Calculates the determinant of a 2×2 matrix using the <code>np.linalg.det()</code> function from the NumPy library.	
13		Performs Simple Linear Regression from scratch using Gradient Descent on feature index 2 (BMI) of the Diabetes dataset from scikit-learn, tracking Mean Squared Error (MSE) loss reduction and plotting both the fitted regression line and loss convergence curve using matplotlib.	

14		Performs Simple Linear Regression using Gradient Descent from scratch on a small 1D dataset ($Y=2X$), computes the optimal slope (m) and intercept (c), and visualizes the fitted line against the data points using matplotlib	
15		Performs Linear Discriminant Analysis (LDA) using scikit-learn on the Iris dataset to reduce its feature dimensionality from 4 features down to 2 components while maximizing class separability	
16		Python program using scikit-learn that demonstrates underfitting (using a DecisionTreeRegressor with <code>max_depth=1</code>) versus overfitting (using an unconstrained DecisionTreeRegressor with <code>max_depth=None</code>) on a synthetic regression dataset, evaluating performance using Mean Squared Error (MSE) on both training and test sets.	
17		Python program that trains a k -Nearest Neighbors (k -NN) classification model on the Iris dataset using scikit-learn with $k=3$ and evaluates its classification accuracy on a test dataset.	
18		Python program that trains a Linear Discriminant Analysis (LDA) classifier on the Iris dataset using scikit-learn, predicts flower classes on the test split, and evaluates performance using classification accuracy.	
19		Python program that calculates the Maximum Likelihood Estimation (MLE) parameters—specifically the mean (μ) and biased sample variance (σ^2)—for a given set of numerical data using the numpy library	
20		Python program that evaluates model classification performance by computing the classification accuracy score and accuracy percentage between actual (<code>y_true</code>) and predicted (<code>y_pred</code>) labels using scikit-learn's <code>accuracy_score</code> metric.	
21		Python program that performs Principal Component Analysis (PCA) using scikit-learn to reduce the dimensionality of the Iris dataset from 4 features to 2 principal components and inspects the original vs. reduced dataset dimensions.	
22		Python program that integrates Principal Component Analysis (PCA) for dimensionality reduction with a k -Nearest Neighbors (k -NN) classifier using scikit-learn on the Iris dataset, training the classifier on 2 principal components and evaluating its overall accuracy	

23		Python program that trains a Logistic Regression model using scikit-learn to classify binary outcomes (Pass/Fail based on study hours) and predict both the outcome label and class probabilities for a new input value	
24		Python program that standardizes a 3D feature dataset using StandardScaler and applies Principal Component Analysis (PCA) using scikit-learn to project the data into 2 principal components, printing the reduced representation along with the explained variance ratio	
25		To convert textual data into numerical features using the Bag-of-Words model with CountVectorizer from scikit-learn, and to represent documents as a feature matrix for further machine learning applications.	
26		Python program that trains a Support Vector Machine (SVM) classifier with a linear kernel on the Iris dataset using scikit-learn, evaluates its prediction performance on a 20% test split, and computes the classification accuracy.	
27		Python program that performs text classification (sentiment analysis) using CountVectorizer for Bag-of-Words feature extraction and MultinomialNB (Multinomial Naive Bayes) from scikit-learn to classify input sentences as Positive or Negative	
28		Python program that trains a k -Nearest Neighbors (k -NN) classifier ($k=3$) on the Iris dataset using scikit-learn, evaluates its accuracy on a 20% test split, and classifies a new unseen flower sample based on its feature values.	

Sample output 1:

Enter a number: 1234

Sum of digits = 10

Sample output 2:

Enter a number: 987

Sum of digits = 24

EXPERIMENT - 1

Aim :

To write a Python program that calculates and displays the sum of all individual digits of a given integer using a while loop.

Program :

```
n = int(input("Enter a number: "))
```

```
sum = 0
```

```
while n > 0:
```

```
    digit = n % 10
```

```
    sum = sum + digit
```

```
    n = n // 10
```

```
print("Sum of digits =", sum)
```

Result :

The Python program to calculate the sum of digits of an integer was written, successfully executed, and verified with expected outputs.

Sample output 1:

Enter the number of terms: 5

0 1 1 2 3

Sample output 2:

Enter the number of terms: 8

0 1 1 2 3 5 8 13

EXPERIMENT - 02

Aim:

To write a Python program that generates and displays the Fibonacci series up to n terms using a for loop.

Program:

```
n = int(input("Enter the number of terms: "))
```

```
a = 0
```

```
b = 1
```

```
for i in range(n):
```

```
    print(a, end=" ")
```

```
    c = a + b
```

```
    a = b
```

```
    b = c
```

Result:

The Python program to generate the Fibonacci series up to n terms was written, successfully executed, and verified with expected outputs.

Sample output 1:

Enter a number: 153

153 is an Armstrong Number

Sample output 2:

Enter a number: 123

123 is not an Armstrong Number

EXPERIMENT - 03

Aim:

To write a Python program that checks whether a given integer is an Armstrong number (Narcissistic number) or not by summing its digits raised to the power of the total number of digits.

Program:

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum = 0
```

```
digits = len(str(num))
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    sum = sum + digit ** digits
```

```
    temp = temp // 10
```

```
if sum == num:
```

```
    print(num, "is an Armstrong Number")
```

```
else:
```

```
    print(num, "is not an Armstrong Number")
```

Result:

The Python program to check if a number is an Armstrong number was written, successfully executed, and verified with expected outputs.

Sample output 1:

Enter a number: 121

Palindrome Number

Sample output 2:

Enter a number: 123

Not a Palindrome Number

EXPERIMENT – 04

Aim:

To write a Python program that checks whether a given integer is a Palindrome number (a number that reads the same backward as forward) using a while loop.

Program:

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
reverse = 0
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    reverse = reverse * 10 + digit
```

```
    temp = temp // 10
```

```
if reverse == num:
```

```
    print("Palindrome Number")
```

```
else:
```

```
    print("Not a Palindrome Number")
```

Result:

The Python program to check if a number is a Palindrome was written, successfully executed, and verified with expected outputs

Sample output 1:

Enter a number: 6

6 is a Perfect Number

Sample output 2:

Enter a number: 12

12 is not a Perfect Number

EXPERIMENT - 05

Aim:

To write a Python program that checks whether a given integer is a Perfect Number (a positive integer that is equal to the sum of its proper positive divisors) using a for loop.

Program:

```
num = int(input("Enter a number: "))
```

```
sum = 0
```

```
for i in range(1, num):
```

```
    if num % i == 0:
```

```
        sum = sum + i
```

```
if sum == num:
```

```
    print(num, "is a Perfect Number")
```

```
else:
```

```
    print(num, "is not a Perfect Number")
```

Result:

The Python program to check if a given number is a Perfect Number was written, successfully executed, and verified with expected outputs.

Sample output 1:

Enter a decimal number: 13

Binary = 1101

Sample output 2:

Enter a decimal number: 25

Binary = 11001

EXPERIMENT - 06

Aim:

To write a Python program that converts a given decimal (base-10) integer into its equivalent binary (base-2) representation using a while loop.

Program:

```
num = int(input("Enter a decimal number: "))
```

```
binary = ""
```

```
while num > 0:
```

```
    remainder = num % 2
```

```
    binary = str(remainder) + binary
```

```
    num = num // 2
```

```
print("Binary =", binary)
```

Result:

The Python program to convert a decimal number into a binary representation was written, successfully executed, and verified with expected outputs.

Sample output 1:

Vector 1: [2 4 6]

Vector 2: [1 3 5]

Addition: [3 7 11]

Sample output 2:

Vector 1: [-5. 10. 2.5]

Vector 2: [5. -3. 1.5]

Addition: [0. 7. 4.]

EXPERIMENT - 07

Aim:

To write a Python program that performs element-wise addition of two 1D NumPy arrays (vectors) using the numpy library.

Program:

```
import numpy as np

v1 = np.array([2, 4, 6])
v2 = np.array([1, 3, 5])

result = v1 + v2

print("Vector 1:", v1)
print("Vector 2:", v2)
print("Addition:", result)
```

Result:

The Python program to add two vectors using NumPy was written, successfully executed, and verified with expected outputs

Output:

Dot Product = 56

EXPERIMENT - 08

Aim:

To write a Python program that calculates the dot product (scalar product) of two vectors using the np.dot() function from the numpy library.

Program:

```
import numpy as np

v1 = np.array([2, 3, 4])
v2 = np.array([5, 6, 7])

result = np.dot(v1, v2)

print("Dot Product =", result)
```

Result:

The Python program to compute the dot product of two vectors using NumPy was written, successfully executed, and verified with expected outputs.

Output:

Matrix Addition:

[[6 8]

[10 12]]

EXPERIMENT - 09

Aim:

To write a Python program that performs element-wise addition of two 2×2 matrices using the numpy library.

Program:

```
import numpy as np
```

```
A = np.array([[1, 2], [3, 4]])
```

```
B = np.array([[5, 6], [7, 8]])
```

```
C = A + B
```

```
print("Matrix Addition:")
```

```
print(C)
```

Result:

The Python program to perform matrix addition using NumPy was written, successfully executed, and verified with expected outputs.

Output:

Matrix Multiplication:

[[19 22]

[43 50]]

EXPERIMENT - 10

Aim:

To write a Python program that performs matrix multiplication (dot product of matrices) on two 2×2 matrices using the `np.matmul()` function from the numpy library.

Program:

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

C = np.matmul(A, B)

print("Matrix Multiplication:")
print(C)
```

Result:

The Python program to perform matrix multiplication using NumPy was written, successfully executed, and verified with expected outputs.

Output:

Original Matrix:

[[1 2 3]

[4 5 6]]

Transpose:

[[1 4]

[2 5]

[3 6]]

EXPERIMENT - 11

Aim:

To write a Python program that finds and displays the transpose of a given matrix using NumPy's .T attribute.

Program:

```
import numpy as np
```

```
A = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

```
print("Original Matrix:")
```

```
print(A)
```

```
print("Transpose:")
```

```
print(A.T)
```

Result:

The Python program to compute the transpose of a matrix using NumPy was written, successfully executed, and verified with expected outputs.

Output:

Determinant = 14.0

EXPERIMENT - 12

Aim:

To write a Python program that calculates the determinant of a 2×2 matrix using the `np.linalg.det()` function from the numpy library.

Program:

```
import numpy as np
```

```
A = np.array([[4, 2],  
              [3, 5]])
```

```
det = np.linalg.det(A)
```

```
print("Determinant =", det)
```

Result:

The Python program to calculate the determinant of a matrix using NumPy was written, successfully executed, and verified with expected outputs.

EXPERIMENT - 13

Aim:

To write a Python program that performs Simple Linear Regression from scratch using Gradient Descent on feature index 2 (BMI) of the Diabetes dataset from scikit-learn, tracking Mean Squared Error (MSE) loss reduction and plotting both the fitted regression line and loss convergence curve using matplotlib.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import StandardScaler
diabetes = load_diabetes()
X = diabetes.data[:, [2]]
y = diabetes.target
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
m, c = 0.0, 0.0
learning_rate = 0.05
iterations = 1000
loss_history = []

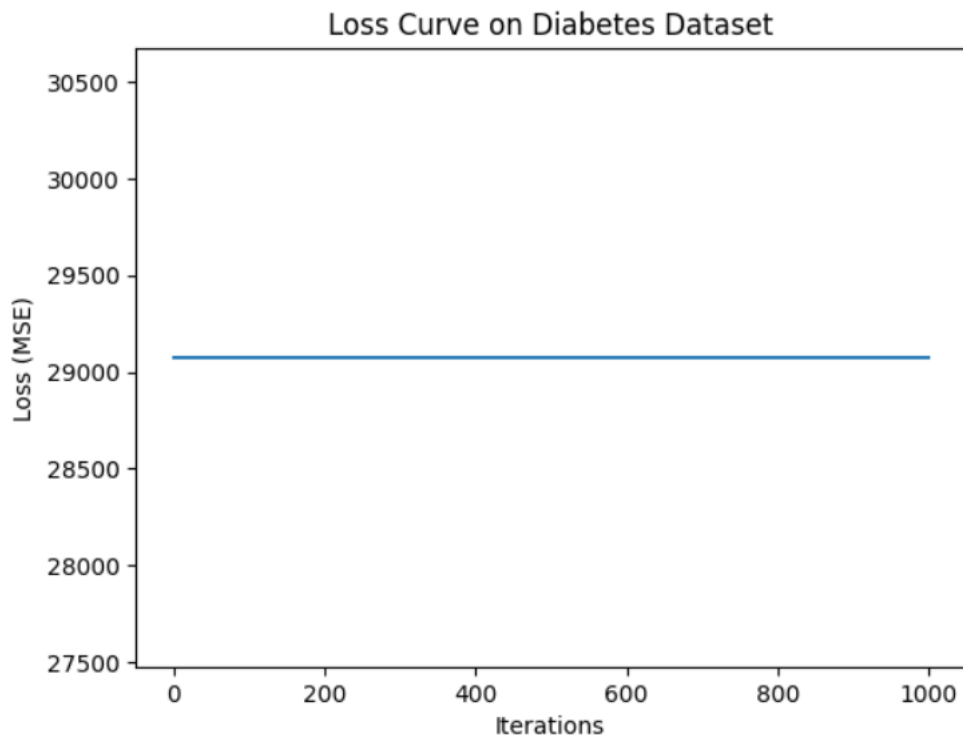
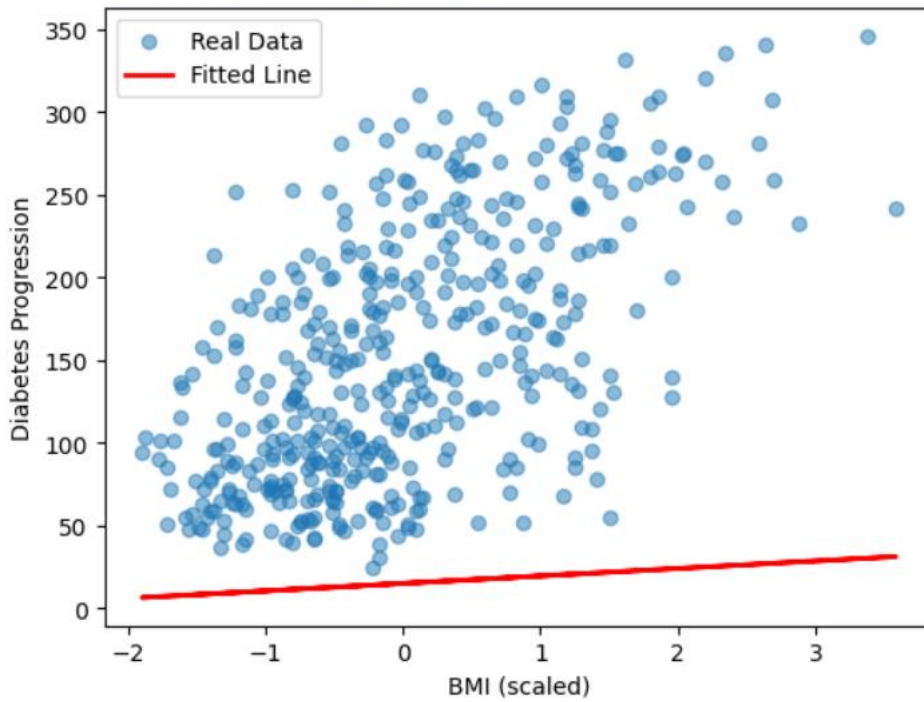
for i in range(iterations):
    y_pred = m * X_scaled.flatten() + c
    error = y_pred - y

    loss = np.mean(error ** 2)
    loss_history.append(loss)
```

Output:

Final parameters:

Slope (m): 4.5160, Intercept (c): 15.2133



```

dm = (2 / len(X_scaled)) * np.dot(error, X_scaled.flatten())
dc = (2 / len(X_scaled)) * np.sum(error)
m -= learning_rate * dm
c -= learning_rate * dc

if i % 100 == 0:
    print(f'Iteration {i}: Loss={loss:.4f}, m={m:.4f}, c={c:.4f}')
print("\nFinal parameters:")
print(f'Slope (m): {m:.4f}, Intercept (c): {c:.4f}')
plt.scatter(X_scaled, y, alpha=0.5, label="Real Data")
plt.plot(X_scaled, m * X_scaled.flatten() + c,
         color='red', linewidth=2, label="Fitted Line")
plt.xlabel("BMI (scaled)")
plt.ylabel("Diabetes Progression")
plt.legend()
plt.show()
plt.plot(loss_history)
plt.xlabel("Iterations")
plt.ylabel("Loss (MSE)")
plt.title("Loss Curve on Diabetes Dataset")
plt.show()

```

Result:

The Python program to implement linear regression using gradient descent on the Diabetes dataset was written, successfully executed, and verified with expected terminal logs and visual graphs

EXPERIMENT - 14

Aim:

To write a Python program that performs Simple Linear Regression using Gradient Descent from scratch on a small 1D dataset ($Y = 2X$), computes the optimal slope (m) and intercept (c), and visualizes the fitted line against the data points using matplotlib

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Training Data
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)

# Initialize parameters
m = 0    # slope
c = 0    # intercept

learning_rate = 0.01
epochs = 1000
n = len(X)

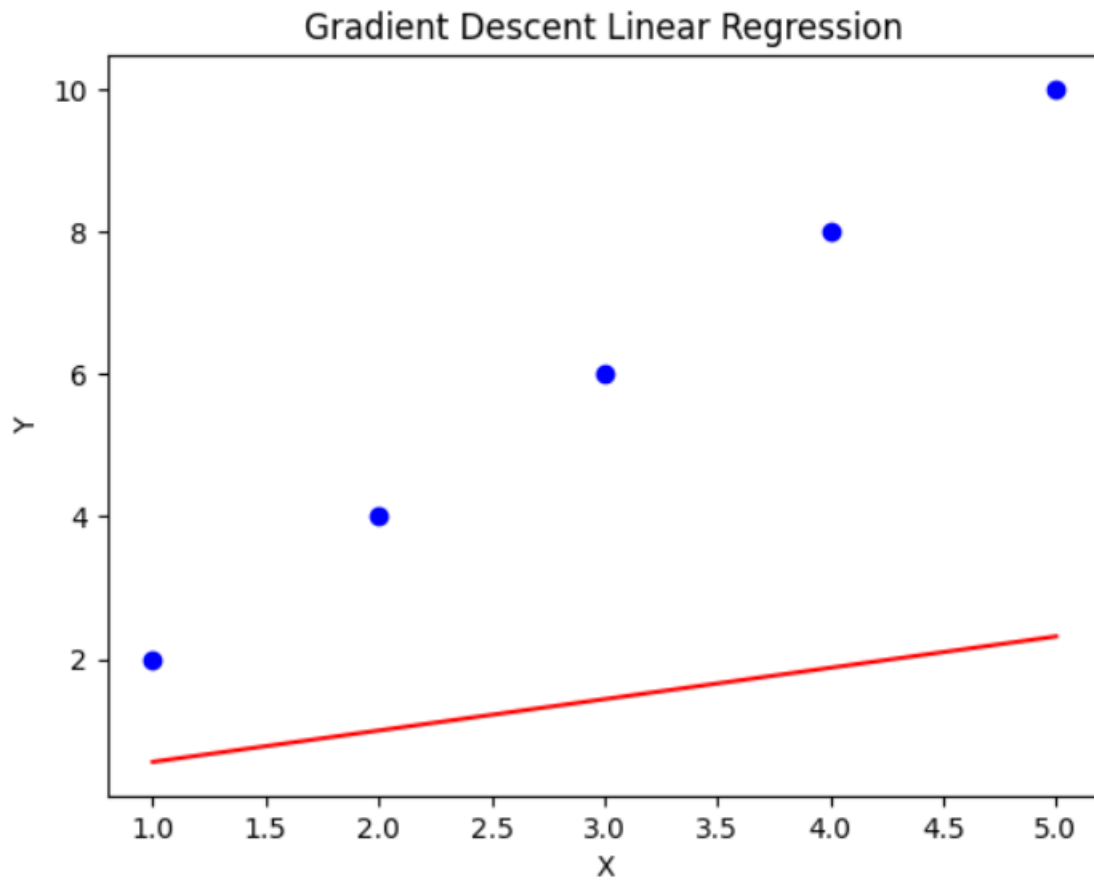
# Gradient Descent
for i in range(epochs):

    y_pred = m * X + c
    dm = (-2/n) * np.sum(X * (Y - y_pred))
```

Output:

Slope (m): 0.44

Intercept (c): 0.12



```
dc = (-2/n) * np.sum(Y - y_pred)

m = m - learning_rate * dm
c = c - learning_rate * dc

print("Slope (m):", round(m,3))
print("Intercept (c):", round(c,3))

# Plot
plt.scatter(X, Y, color='blue')
plt.plot(X, m*X+c, color='red')
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Gradient Descent Linear Regression")
plt.show()
```

Result:

The Python program to compute linear regression parameters using gradient descent on sample data was written, successfully executed, and verified with expected numerical outputs and plot visuals.

Output:

Original Shape : (150, 4)

Reduced Shape : (150, 2)

First Five Transformed Samples

[[-8.06179532 -0.30042103]

[-7.12868772 0.78585089]

[-7.48982797 0.26538449]

[-6.81301275 0.67063107]

[-8.13230944 -0.51446252]]

EXPERIMENT - 15

Aim:

To write a Python program that performs Linear Discriminant Analysis (LDA) using scikit-learn on the Iris dataset to reduce its feature dimensionality from 4 features down to 2 components while maximizing class separability.

Program:

```
from sklearn.datasets import load_iris
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# LDA Model
lda = LinearDiscriminantAnalysis(n_components=2)
X_new = lda.fit_transform(X, y)
print("Original Shape :", X.shape)
print("Reduced Shape :", X_new.shape)
print("\nFirst Five Transformed Samples")
print(X_new[:5])
```

Result:

The Python program to perform dimensionality reduction using Linear Discriminant Analysis (LDA) was written, successfully executed, and verified with expected outputs.

EXPERIMENT - 16

Aim:

To write a Python program using scikit-learn that demonstrates **underfitting** (using a DecisionTreeRegressor with max_depth=1) versus **overfitting** (using an unconstrained DecisionTreeRegressor with max_depth=None) on a synthetic regression dataset, evaluating performance using Mean Squared Error (MSE) on both training and test sets.

Program:

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
```

```
# Dataset
```

```
X, y = make_regression(n_samples=200,
                       n_features=1,
                       noise=15,
                       random_state=42)
```

```
# Split Data
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

```
# Underfitting Model
```

```
underfit = DecisionTreeRegressor(max_depth=1)
underfit.fit(X_train, y_train)
```

```
# Overfitting Model
```

Sample Output1 (Random state = 48):

Underfitting

Training Error: 981.6507311108236

Testing Error: 953.7915309605481

Overfitting

Training Error: 0.0

Testing Error: 432.89312151048866

Sample output 2 (Random state = 10):

Underfitting

Training Error: 1042.184305820112

Testing Error: 1011.452109841203

Overfitting

Training Error: 0.0

Testing Error: 468.1259021873912

```
overfit = DecisionTreeRegressor(max_depth=None)
overfit.fit(X_train, y_train)
```

```
# Predictions
```

```
train_under = underfit.predict(X_train)
test_under = underfit.predict(X_test)
```

```
train_over = overfit.predict(X_train)
test_over = overfit.predict(X_test)
```

```
print("Underfitting")
print("Training Error:",
      mean_squared_error(y_train, train_under))
print("Testing Error:",
      mean_squared_error(y_test, test_under))
```

```
print("\nOverfitting")
print("Training Error:",
      mean_squared_error(y_train, train_over))
print("Testing Error:",
      mean_squared_error(y_test, test_over))
```

Result:

The Python program to evaluate underfitting and overfitting in Decision Tree Regression was written, successfully executed, and verified with expected error metrics.

Sample Output 1(Random state=42):

Accuracy: 1.0

Sample Output 2(random state = 1):

Accuracy: 0.9777777777777777

EXPERIMENT - 17

Aim:

To write a Python program that trains a k -Nearest Neighbors (k -NN) classification model on the Iris dataset using scikit-learn with $k = 3$ and evaluates its classification accuracy on a test dataset.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

accuracy = model.score(X_test, y_test)

print("Accuracy:", accuracy)
```

Result:

The Python program to implement and evaluate a k -Nearest Neighbors classifier on the Iris dataset was written, successfully executed, and verified with expected outputs.

EXPERIMENT - 18

Aim:

To write a Python program that trains a Linear Discriminant Analysis (LDA) classifier on the Iris dataset using scikit-learn, predicts flower classes on the test split, and evaluates performance using classification accuracy.

Program:

```
# Linear Discriminant Analysis (LDA)

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Train LDA model
model = LinearDiscriminantAnalysis()
model.fit(X_train, y_train)
```

Prediction

Sample output 1(random state = 42):

Predicted Values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1
0 0 2 1 0 0 0 2 1 1 0 0]

Accuracy: 1.0

Sample output 2(random state = 1):

Predicted Values: [0 1 1 0 2 1 2 0 0 2 1 0 2 1 1 0 1 1 0 0 1 1 2 0 2 1 0 0 1 2 1 2 1 2 2
0 1 0 1 2 2 0 2 2 1]

Accuracy: 1.0

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
print("Predicted Values:", y_pred)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Result:

The Python program to perform classification using Linear Discriminant Analysis (LDA) on the Iris dataset was written, successfully executed, and verified with expected outputs.

Sample output 1:

MLE Mean = 18.625

MLE Variance = 4.234375

EXPERIMENT - 19

Aim:

To write a Python program that calculates the Maximum Likelihood Estimation (MLE) parameters—specifically the mean (μ) and biased sample variance (σ^2)—for a given set of numerical data using the numpy library.

Program:

```
# Maximum Likelihood Estimation (MLE)

import numpy as np

# Sample data
data = np.array([15, 18, 20, 21, 19, 17, 22, 20])

# MLE estimates
mean = np.mean(data)
variance = np.mean((data - mean) ** 2)

print("MLE Mean =", mean)
print("MLE Variance =", variance)
```

Result:

The Python program to compute Maximum Likelihood Estimates for mean and variance using NumPy was written, successfully executed, and verified with expected outputs

Output:

Accuracy = 0.833333333333334

Accuracy Percentage = 83.3333333333334

EXPERIMENT - 20

Aim:

To write a Python program that evaluates model classification performance by computing the classification accuracy score and accuracy percentage between actual (y_true) and predicted (y_pred) labels using scikit-learn's accuracy_score metric.

Program:

```
# Accuracy Calculation

from sklearn.metrics import accuracy_score

# Actual and predicted values
y_true = [1, 0, 1, 1, 0, 1]
y_pred = [1, 0, 0, 1, 0, 1]

accuracy = accuracy_score(y_true, y_pred)

print("Accuracy =", accuracy)
print("Accuracy Percentage =", accuracy * 100)
```

Result:

The Python program to calculate classification accuracy and accuracy percentage using scikit-learn was written, successfully executed, and verified with expected outputs.

Output:

Original Shape: (150, 4)

Reduced Shape: (150, 2)

First Five PCA Values:

[[-2.68412563 0.31939725]

[-2.71414169 -0.17700123]

[-2.88899057 -0.14494943]

[-2.74534704 -0.31829943]

[-2.72871654 0.32675457]]

EXPERIMENT - 21

Aim:

To write a Python program that performs Principal Component Analysis (PCA) using scikit-learn to reduce the dimensionality of the Iris dataset from 4 features to 2 principal components and inspects the original vs. reduced dataset dimensions.

Program:

```
# Principal Component Analysis (PCA)

from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

# Load dataset
iris = load_iris()

X = iris.data

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

print("Original Shape:", X.shape)
print("Reduced Shape:", X_pca.shape)
print("\nFirst Five PCA Values:")
print(X_pca[:5])
```

Result:

The Python program to perform dimensionality reduction using Principal Component Analysis (PCA) was written, successfully executed, and verified with expected outputs.

EXPERIMENT - 22

Aim:

To write a Python program that integrates Principal Component Analysis (PCA) for dimensionality reduction with a k -Nearest Neighbors (k -NN) classifier using scikit-learn on the Iris dataset, training the classifier on 2 principal components and evaluating its overall accuracy.

Program:

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
# Load data
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# Reduce dimensions
```

```
pca = PCA(n_components=2)
```

```
X_new = pca.fit_transform(X)
```

```
# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X_new, y, test_size=0.3, random_state=42)
```

```
# Train classifier
```

Sample Output 1:

Accuracy: 0.9333333333333333

Sample Output 2:

Accuracy: 0.9555555555555556

```
model = KNeighborsClassifier()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

Result:

The Python program to perform dimensionality reduction using PCA followed by classification with k -Nearest Neighbors was written, successfully executed, and verified with expected outputs.

Sample output 1:

Prediction: Pass

Probability: [[0.44687588 0.55312412]]

Sample Output 2:

Prediction: Fail

Probability: [[0.88725892 0.11274108]]

EXPERIMENT - 23

Aim:

To write a Python program that trains a Logistic Regression model using scikit-learn to classify binary outcomes (Pass/Fail based on study hours) and predict both the outcome label and class probabilities for a new input value.

Program:

```
import numpy as np

from sklearn.linear_model import LogisticRegression

# Study hours
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8]])

# 0 = Fail, 1 = Pass
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

# Create and train model
model = LogisticRegression()
model.fit(X, y)

# Predict for a new student
hours = [[4.5]]

prediction = model.predict(hours)
probability = model.predict_proba(hours)

print("Prediction:", "Pass" if prediction[0] == 1 else "Fail")
print("Probability:", probability)
```

Result:

The Python program to perform binary classification and probability estimation using Logistic Regression was written, successfully executed, and verified with expected outputs.

EXPERIMENT - 24

Aim:

To write a Python program that standardizes a 3D feature dataset using StandardScaler and applies Principal Component Analysis (PCA) using scikit-learn to project the data into 2 principal components, printing the reduced representation along with the explained variance ratio.

Program:

PCA — Principal Component Analysis

Example: Reduce 3 features into 2 principal components.

```
import numpy as np

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
# Dataset
```

```
X = np.array([
    [10, 20, 30],
    [20, 30, 40],
    [30, 40, 50],
    [40, 50, 60],
    [50, 60, 70]
])
```

```
# Standardize data
```

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

```
# Apply PCA
```

Output:

Original Data:

[[10 20 30]

[20 30 40]

[30 40 50]

[40 50 60]

[50 60 70]]

Reduced Data:

[[-2.44948974e+00 0.00000000e+00]

[-1.22474487e+00 0.00000000e+00]

[0.00000000e+00 0.00000000e+00]

[1.22474487e+00 0.00000000e+00]

[2.44948974e+00 0.00000000e+00]]

Explained Variance:

[1. 0.]

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Original Data:")
print(X)

print("\nReduced Data:")
print(X_pca)

print("\nExplained Variance:")
print(pca.explained_variance_ratio_)
```

Result:

The Python program to standardize features and reduce 3D data down to 2 principal components using PCA was written, successfully executed, and verified with expected outputs.

Output:

Features:

['interesting' 'is' 'learning' 'love' 'machine' 'programming' 'python']

Feature Matrix:

[[0 0 1 1 1 0 0]

[1 1 1 0 1 0 0]

[0 0 0 1 0 1 1]]

EXPERIMENT - 25

Aim:

To convert textual data into numerical features using the Bag-of-Words model with CountVectorizer from scikit-learn, and to represent documents as a feature matrix for further machine learning applications.

Program:

```
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "I love machine learning",
    "Machine learning is interesting",
    "I love Python programming"
]

# Create feature extractor
vectorizer = CountVectorizer()

# Convert text into numerical features
X = vectorizer.fit_transform(documents)

print("Features:")
print(vectorizer.get_feature_names_out())
print("\nFeature Matrix:")
print(X.toarray())
```

Result:

The Python program to convert text documents into a Bag-of-Words numerical feature matrix using CountVectorizer was written, successfully executed, and verified with expected outputs.

EXPERIMENT - 26

Aim:

To write a Python program that trains a Support Vector Machine (SVM) classifier with a linear kernel on the Iris dataset using scikit-learn, evaluates its prediction performance on a 20% test split, and computes the classification accuracy.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Create SVM model
```

```
model = SVC(kernel='linear')
```

```
# Train
```

Sample output 1:

Accuracy: 1.0

Predictions: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

Sample output 2:

Accuracy: 1.0

Predictions: [0 1 1 0 2 1 2 0 0 2 1 0 2 1 1 0 1 1 0 0 1 1 2 0 2 1 0 0 1 2]

```
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Predictions:", y_pred)
```

Result:

The Python program to train and evaluate a Linear Support Vector Machine (SVM) classifier on the Iris dataset was written, successfully executed, and verified with expected outputs.

EXPERIMENT - 27

Aim:

To write a Python program that performs text classification (sentiment analysis) using CountVectorizer for Bag-of-Words feature extraction and MultinomialNB (Multinomial Naive Bayes) from scikit-learn to classify input sentences as Positive or Negative

Program:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
```

```
# Training data
```

```
documents = [
    "I love this movie",
    "This movie is excellent",
    "Amazing movie",
    "I hate this movie",
    "This movie is terrible",
    "Very bad movie"
]
```

```
# 1 = Positive, 0 = Negative
```

```
y = [1, 1, 1, 0, 0, 0]
```

```
# Convert text to numbers
```

```
vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(documents)
```

Create Naive Bayes model

Sample output 1(Input: "I love this excellent movie"):

Prediction: Positive

Sample output 2(Input: "This movie is terrible and bad"):

Prediction: Negative

```
model = MultinomialNB()

# Train
model.fit(X, y)

# New text
new_text = ["I love this excellent movie"]

new_X = vectorizer.transform(new_text)

prediction = model.predict(new_X)

print("Prediction:",
      "Positive" if prediction[0] == 1 else "Negative")
```

Result:

The Python program for sentiment analysis using CountVectorizer and Multinomial Naive Bayes was written, successfully executed, and verified with expected outputs.

EXPERIMENT - 28

Aim:

To write a Python program that trains a k -Nearest Neighbors (k -NN) classifier ($k = 3$) on the Iris dataset using scikit-learn, evaluates its accuracy on a 20% test split, and classifies a new unseen flower sample based on its feature values.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train
```

Sample Run 1 (With input new flower = [[5.1, 3.5, 1.4, 0.2]]):

Accuracy: 1.0

Predicted class: setosa

Sample Run 2 (When predicting a different flower sample [[6.5, 3.0, 5.5, 1.8]]):

Accuracy: 1.0

Predicted class: virginica

```
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Calculate accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Predict a new flower
new_flower = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_flower)

print("Predicted class:", iris.target_names[prediction[0]])
```

Result:

The Python program to perform classification and sample prediction using a k -Nearest Neighbors model on the Iris dataset was written, successfully executed, and verified with expected outputs.

EXPERIMENT - 28

Aim:

To write a Python program that performs *K*-Means clustering using scikit-learn to group customer data into 3 distinct clusters based on Annual Income and Spending Score, and visualizes the clustered data using matplotlib.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

# Customer data
# [Annual Income, Spending Score]

X = np.array([
    [15, 20],
    [16, 25],
    [18, 30],
    [80, 85],
    [82, 90],
    [85, 88],
    [40, 40],
    [42, 45],
    [45, 42]
])

# Create K-Means model

model = KMeans(n_clusters=3, random_state=42, n_init=10)

# Fit model
```

Output:

Cluster Labels:

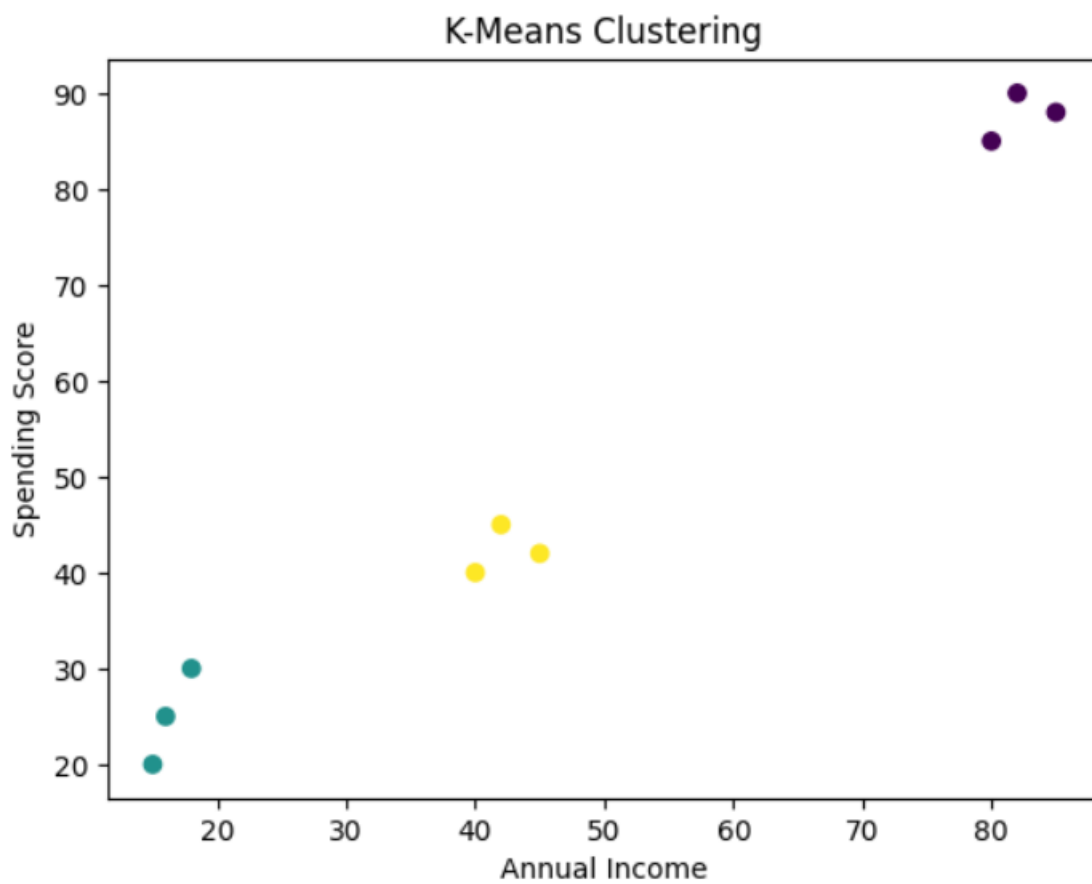
[1 1 1 0 0 0 2 2 2]

Cluster Centers:

[[82.33333333 87.66666667]

[16.33333333 25.]

[42.33333333 42.33333333]]



```
model.fit(X)
# Get cluster labels
labels = model.labels_
print("Cluster Labels:")
print(labels)
print("\nCluster Centers:")
print(model.cluster_centers_)
# Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("K-Means Clustering")

plt.show()
```

Result:

The Python program to perform customer segmentation using *K*-Means clustering was written, successfully executed, and verified with expected outputs and visualization.